

From Language to Logic: A Theoretical Architecture for VLM-Grounded Safe Navigation

Kristy Sakano¹, Kalonji Harrington¹, and Huan Xu²

Abstract—We propose an architecture for integrating high-level, human-provided safety rules and operator-aligned semantic preferences into autonomous robot navigation in unstructured outdoor environments. In our approach, natural-language rules are translated into Signal Temporal Logic (STL) specifications that guide planning and navigation during runtime. Persistent, environment-centric rules and terrain preferences are grounded into a 2D cost map, while temporally dynamic requirements are expressed as STL specifications to be monitored during runtime. We hypothesize the use of Vision-Language Models (VLMs) for zero-shot scene understanding, enabling mapping between human instructions, semantic features, and environmental constraints. Within this framework, we construct an illustrative navigation model that is designed to satisfy a set of STL-encoded specifications and soft operator preferences through formal satisfaction metrics embedded into environmental properties and runtime monitoring.

Index Terms—autonomy, levels of safety, navigation

I. INTRODUCTION

Uncrewed Aerial Vehicles (UAVs) and other autonomous mobile robots are increasingly deployed in environments that may be novel or only partially known at deployment time [1]–[3]. Path planning is central to these deployments: given a representation of the environment, the planner must construct a path from the robot’s current location to a goal while avoiding obstacles. For scenarios requiring high safety considerations, such as platforms operating in close proximity to humans, there is an additional requirement that the trajectory satisfy strict safety constraints and align with operating rules [4]. In Figure 1, these rules may be time-invariant, like specification ϕ_1 “Always maintain 2 meter clearance from trees,” or time-varying, like ϕ_3 “Exit grass within 5 seconds upon entrance.” These rules motivate an evaluation that tracks rule satisfaction, violation, and margin along a trajectory, rather than only reporting aggregate success rates.

Ensuring safe navigation in real-world environments requires perceiving not only the geometry of surrounding obstacles, but also their semantic meaning [5]. For instance, traversing grass, water, or pavement may demand distinct behaviors depending on the associated safety risks to the robot and its surroundings. Classical navigation stacks primarily rely on dense sensing (LiDAR, depth cameras, IMUs)

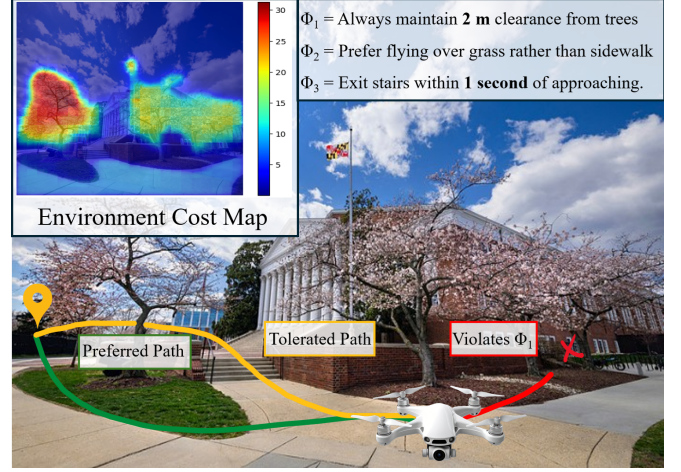


Fig. 1. Autonomous robot navigation under the proposed theoretical architecture. Operator-defined safety rules are translated into Signal Temporal Logic (STL) specifications ϕ_1, ϕ_2, ϕ_3 with time-invariant preferences embedded into the environment cost map and time-varying rules integrated into the planning component and monitored at runtime.

to reconstruct spatial maps and detect obstacles [6]. However, these systems often depend on high-fidelity maps and accurate self-localization, making them brittle in unknown or unstructured environments. When using monocular, vision-based navigation, the challenge shifts to maintaining comparable safety standards with less geometric information. To navigate safely, robots must interpret the environment with semantic perception: identifying traversable regions and distinguishing objects in their environment. Classical navigation stacks are built around geometric maps and hand-engineered perception modules, so expressing rules such as “Avoid flying over red flowers, but permit flying over other flowers,” typically require thresholds for color or task-specific segmentation models, and do not generalize well beyond predefined classes [7], [8].

To provide this level of semantic perception on vision-based platforms, researchers have begun to explore Vision-Language Models (VLMs) that extract rich scene understanding directly from RGB observations [9], [10]. These models combine visual input and natural language, using both to jointly identify and reason about semantically meaningful objects in an image based on high-level language instructions. Recently introduced VLMs can also provide zero-shot scene understanding, extracting useful semantic information from raw RGB images without requiring exhaustive task-specific labeling. In the context of navigation, VLMs enable open-

¹Department of Computer Science, University of Maryland, College Park, USA. Email: kvsakano@umd.edu and kharring@umd.edu

²Department of Aerospace Engineering, University of Maryland, College Park, USA. Email: mumu@umd.edu

This work has been accepted for publication in the proceedings of the International Conference on Unmanned Aircraft Systems (ICUAS), 2026.

vocabulary semantic segmentation and semantic mapping, allowing operators to describe relevant regions and objects in natural language and to ground these descriptions in the robot’s visual observations. However, while VLMs expand what the robot can recognize and describe, they do not by themselves guarantee that the resulting behavior satisfies critical safety requirements.

In parallel, formal methods have been widely used to specify and verify autonomous robot behavior, from correct-by-construction motion planning with temporal logic specifications, to runtime monitoring using Signal Temporal Logic (STL) [11]. These approaches provide guarantees over complex behaviors and have been applied to both classical controllers and learning-based policies, where temporal logic bounds or evaluates behavior [12], [13].

Despite these complementary strengths, most existing temporal logic-based specification and verification methods rely on hand-crafted predicates and pre-defined region labels, and therefore do not directly ground human language rules in raw image-based observations. Conversely, VLM-based navigation can express open-vocabulary semantic concepts but lack formalism to ensure adherence to safety constraints over time. This disconnect motivates combining temporal logic-based runtime assurance with modern VLMs for RGB-based navigation, enforcing safety rules over time in visually complex, novel environments.

Main Contributions: This work integrates formal evaluation into an RGB-based navigation architecture to enforce adherence to human-specified safety rules during both planning and runtime. We instantiate this framework with a theoretical navigation stack that integrates a Vision-Language Model (VLM) with Signal Temporal Logic (STL) specifications to provide runtime assurance for autonomous robots operating in unstructured outdoor environments.

1. STL-based integration of human rules into navigation:

We translate operator-provided natural language rules into STL specifications and enforce them at two levels of the navigation stack. Time-invariant, environment-centric rules are grounded into a 2D cost map, while time-varying rules are used both for planning-time screening of candidate paths (offline robustness evaluation) and for execution-time monitoring of the candidate path (online monitoring). This dual-layer structure enables the planner to reason over both time-varying and time-invariant requirements within a unified framework.

2. VLM grounding of formal specifications in novel environments:

We use a VLM to semantically interpret outdoor environments, applying open-vocabulary segmentation to identify relevant objects and regions described in human-provided rules. The VLM’s semantic outputs encode both hard spatial constraints in the cost map and environmental properties that affect the robot’s state. By grounding STL predicates directly in visual observations, the robot can reason about which regions are safe to enter or avoid, even in novel environments.

3. Runtime assurance under changing safety requirements:

We outline a theoretical demonstration in an outdoor

scenario with two operational modes: a *normal mode* with baseline safety specifications and a degraded *low-battery mode* with stricter requirements on speed limits, obstacle clearance, and time-bounded behavior. A mid-episode switch from the normal to degraded mode triggers recomputation of the environment cost map and re-planning of a new STL-compliant path. This scenario illustrates how tightening safety constraints and changing robot states can be handled systematically at runtime while still providing formal guarantees for autonomous navigation.

II. BACKGROUND

Ensuring that robots act safely remains a fundamental challenge in robotics, particularly when rules are required to be followed and not embedded as only soft preferences. In this work, we focus on VLM-grounded navigation of an autonomous robot in novel environments.

We focus on a classical planning-based navigation architecture, thereby sidestepping some of the additional challenges associated with safety specification and verification of learning-based policies. Limitations of learning-based systems however, can be addressed by applying formal methods to policy synthesis, such as certificate-based verification methods, or policy verification, such as reachability analysis [14]. Our work is also distinct from Vision-and-Language Navigation, an extension of VLM to the robotics domain in which autonomous robots learn to follow natural language instructions to navigate novel environments. Our architecture does not align with the canonical definition of VLN, in which VLMs participate in action selection to navigate based on natural language goals. Nevertheless, the proposed framework can be layered on top of existing VLN or other vision-navigation systems as it is a safety and evaluation layer that is agnostic to the underlying policy.

In this section, we review relevant literature on (1) VLMs, (2) vision-based navigation in novel environments, and (3) the application of formal verification in autonomous robot navigation. Table I shows a comparison between current implementations of VLNs, VLM-based navigation architectures, and non-VLM navigation architectures in the literature.

A. Vision-Language Models (VLMs)

Recent advances in vision-language modeling have produced VLMs that serve as strong foundation models for solving a variety of text and image tasks [24], [25]. These models either jointly learn text and image encoders to align their embeddings [9], [10] or map image inputs to a pretrained Large Language Model (LLM) [26], [27]. Similar to LLMs, their broad pre-training dataset and multi-modal objective allows the model to incorporate rich semantic relationships critical for downstream tasks.

One application well suited for VLMs is open-vocabulary semantic segmentation, which identifies and assigns portions of an image to arbitrary semantic classes, usually at the pixel level [28], [29]. Using the strong semantic link between language and images found in pre-trained VLMs,

Method	VLM	Sensor Type	Plan Verification	Runtime Verification	Planner Type
LM-Nav [15]	Recognition	Camera	None	None	Learning-based
VL-Nav [16]	Detection & Segmentation	Camera & LiDAR	None	None	Classical
Behav [17]	Segmentation & Goal	Camera & LiDAR	None	None	Classical
DV-VLM [18]	Action Selection	Camera	VLM	None	None
Baird and Coogan [19]	None	State signals	STL (finite horizon)	STL (finite horizon)	Classical
Zero-Shot STL [20]	None	State (abstract)	STL (task spec)	None	Learning-based
STL PI Planner [21]	None	Continuous state	STL (traj. cost)	None	Classical
PASTEL [22]	None	State features	STL (finite horizon)	None	Learning-based
S-MSP [23]	None	Multiple Cameras	STL (finite horizon)	None	Learning-based
Ours	Segmentation	Camera	STL (env. cost map)	STL (finite horizon)	Classical

TABLE I

COMPARISON BETWEEN OUR ARCHITECTURE AND REPRESENTATIVE NAVIGATION APPROACHES. COLUMNS INDICATE THE ROLE OF VLMs, SENSING MODALITY, WHETHER STL OR OTHER FORMAL VERIFICATION IS USED AT PLANNING OR RUN TIME, AND PLANNER TYPE.

recent methods have demonstrated impressive generalization to unseen categories and complex, open-world scenes. [30]–[32]. In our setting, we use open-vocabulary segmentation to derive semantic labels and associated costs in a 2D environment map, bridging language-specified rules and the vision-based navigation stack.

B. Vision-based navigation in novel environments

The success of vision-language models in scene understanding has motivated their use in autonomous navigation [9], [24], [25], particularly in visually complex or previously unseen environments. Prior work has incorporated deep semantic segmentation and metric-semantic mapping to identify traversable terrain and obstacles from RGB images, enabling navigation in novel outdoor settings [33], [34]. In parallel, recent systems incorporate VLMs to bias classical planners, construct semantic maps, and guide exploration by highlighting semantically relevant regions, even when the goal is not directly visible [16], [35].

For example, VLM-RRT uses VLM-derived semantic knowledge to bias sampling in a top-down map view, accelerating convergence of sampling-based planners while assuming full observability and focusing primarily on obstacle avoidance [36], [37]. Other systems fuse VLM-based semantic embeddings directly into 3D maps to ground high-level concepts in the navigation representation, thereby supporting goal-directed navigation based on semantic landmarks [33]. These methods demonstrate the utility of VLMs for vision-based navigation in novel environments, but they typically encode safety behavior implicitly through time-invariant cost shaping or heuristics and lack explicit temporal reasoning or formal guarantees of rule compliance.

C. Formal verification of autonomous navigation

Formal methods provide a complementary perspective by specifying desired robot behaviors either during controller synthesis or at runtime [11]. Temporal logic and related verification tools enable reasoning about the evolution of robot behavior over time. In particular, STL is an expressive

formalism for specifying temporal properties of continuous-time, real-valued signals [38]. STL specifications combine atomic predicates, Boolean operators, and bounded temporal operators such as globally (G), until (U), and eventually (F) to express mathematically precise specifications of control systems [39], [40]. Unlike Linear Temporal Logic (LTL), which is evaluated over discrete state sequences, STL is interpreted over dense-time systems and is therefore well suited for analyzing safety and performance in cyber-physical systems.

Prior work has employed STL for runtime assurance by formulating a finite-horizon optimization problem that minimally adjusts the nominal input while maintaining satisfaction of an STL safety specification [19]. More recent work has incorporated STL into trajectory planning with formal satisfaction guarantees. For example, Liu et. al (2025) proposed a hierarchical, data-driven framework that decomposed STL tasks into timed sub-goals and used a diffusion model to stitch together feasible trajectory segments that satisfy the specification [20].

Complementary sampling-based methods integrate STL robustness directly as a cost in model predictive path integral control, solving finite-horizon optimal control problems with STL objectives [21]. In parallel, end-to-end architectures have emerged that map raw perceptual inputs and STL specifications directly to trajectories using transformers with STL-structured conditioning [22], [23]. Across these methods, predicates are typically defined over structured state spaces with fixed region labels; they do not address grounding temporal logic predicates directly in open-vocabulary visual observations, nor do they explicitly separate hard safety constraints from soft operator preferences.

III. APPROACH

In this section, we outline the methodology of our approach, structured into four components: (1) operator-provided hard constraints and soft preferences, (2) VLM-grounded navigation with hard constraints, (3) state change, and (4) Test and Evaluation (T&E) roadmap.

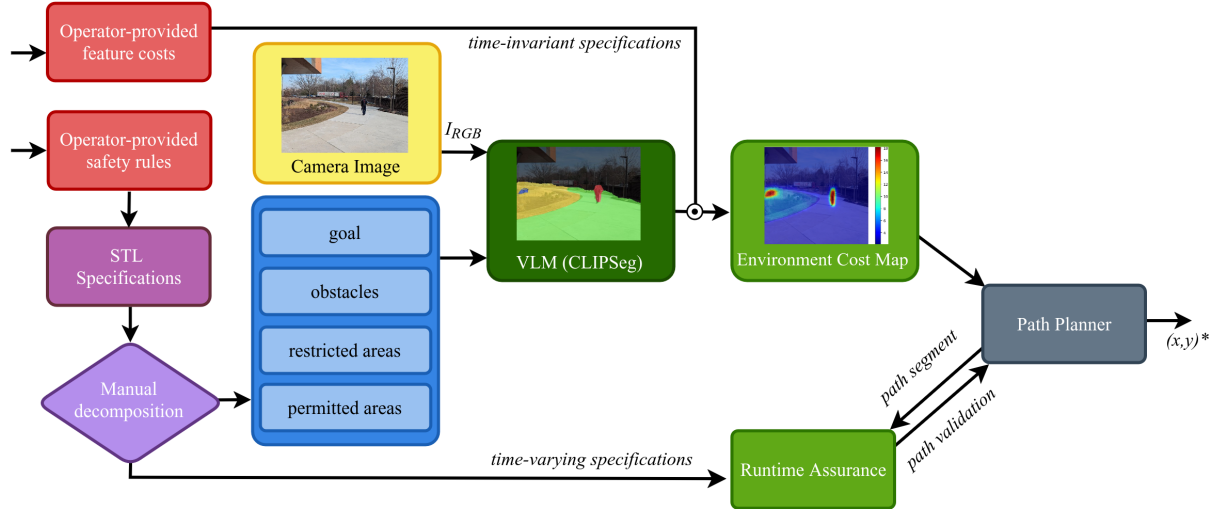


Fig. 2. Overall theoretical architecture of our VLM-grounded safe navigation stack. We obtain *operator-provided feature costs* and *operator-provided safety rules* from mission specifications and hand-translate the rules into *STL specifications* that are manually decomposed into relevant features for the VLM to parse in a *RGB camera image*. Combined with the *feature costs*, we construct an **environment cost map** for all temporally-invariant specifications. The time-varying specifications are incorporated into a **runtime assurance** layer that iteratively validates paths proposed by the *path planner*.

A. Operator-Provided Hard Constraints and Soft Preferences

Our architecture assumes that operators specify both hard constraints and soft preferences prior to runtime. Hard constraints represent safety-critical requirements that must not be violated (e.g., speed limits, obstacle clearance, time-bounded behavior) while soft preferences encode non-critical choices (e.g., preferring flight over grass rather than above sidewalks) [41]. Soft preferences are embedded as static costs in the environment cost map, and hard constraints are encoded both in the cost map (for time-invariant spatial rules) and in STL specifications used during planning and runtime.

1) *Hard Constraints to STL Specifications*: Human operators oversee verification and operation of autonomous robots and ensure compliance with safety standards prior to deployment. Natural language descriptions of hard constraints are translated into STL specifications; the details of this translation process are abstracted from this paper, and we assume the correctness of the translation. Related work demonstrates how natural language processing techniques can be used for transformations [42]–[45].

STL is chosen for its ability to capture temporal relations among continuous system signals, an essential property for autonomous robots that continuously generate rich sensory data streams [38], [39]. Furthermore, STL’s quantitative semantics allow for graded assessment of specification satisfaction or violation via robustness metrics [40]. Unlike aggregate metrics such as the percentage of time spent in hazardous regions, which ignore when and how unsafe states occur, STL specifications explicitly encode ordering, timing, and duration constraints. For example, Rule 3 in Table II, the “*Sidewalk-Exit Temporal Rule*”, requires the robot to exit grass within 5 seconds of entry, permitting frequent but short excursions into grass, while penalizing prolonged grass

traversal.

2) *Embedding Soft Preferences*: Soft operator constraints are embedded into the environment cost map as semantic costs. For example, Rule 5 in Table II, “*Preference to avoid pedestrian sidewalks*”, specifies that the operator prefers flight paths over grass rather than above sidewalks. This preference is implemented as a relative cost difference between grass and sidewalk labels and does not appear as a hard STL constraint.

B. VLM-Grounded Navigation with Hard Constraints

Our approach embeds the STL specifications into the navigation stack in two sections:

1. **Persistent, time-invariant rules** are enforced through spatial representations in the environment cost map,
2. **Dynamic, time-varying rules** are used to evaluate candidate plans produced by the path planner and also continuously monitored during runtime to verify that the evolving system behavior remains within acceptable boundaries (e.g., speed limits, time-bounded responses to environmental conditions), as illustrated in prior works [19], [46].

1) *Environment Cost Map*: We encode time-invariant spatial rules and soft operator preferences in a 2D environment cost map by combining VLM-based semantic segmentation with operator-provided semantic costs. In particular, an open-vocabulary segmentation model such as CLIPSeg [47] produces, for each image pixel, a vector of probabilities over the semantic labels referenced in the operator rules (e.g., *water*, *tree*, *grass*, or *sidewalk*).

Let $\mathcal{I} = \{1, \dots, H\} \times \{1, \dots, W\}$ denote the image grid, and let $\mathcal{L} = \{\ell_1, \dots, \ell_n\}$ denote the set of all semantic labels relevant to motion planning and our rules/preferences. We define the pixel-wise semantic segmentation function

$$\text{Seg} : \mathcal{I} \rightarrow \mathbb{R}^n, \quad (1)$$

where $\text{Seg}(i, j)$ is a length- n vector whose k -th component gives the confidence that pixel (i, j) belongs to semantic label $\ell_k \in \mathcal{L}$.

We can stack these per-pixel outputs into a tensor

$$\mathbf{S} \in \mathbb{R}^{H \times W \times n}, \quad (2)$$

where S_{ijk} denotes the k -th component of $\text{Seg}(i, j)$. Thus, $\text{Seg}(i, j)$ and $\mathbf{S}$ represent the same semantic information at two different levels of notation: $\text{Seg}(i, j)$ refers to the semantic label vector at a single pixel, while $\mathbf{S}$ denotes the full segmentation tensor over the image.

Let $\mathbf{c} \in \mathbb{R}_{\geq 0}^n$ denote the vector of operator-provided costs associated with the semantic classes in $\mathcal{L}$. These costs encode both hard spatial penalties and soft terrain preferences. For example, *obstacle* or *tree* labels may be assigned very high costs, while *grass* may be assigned a lower cost than *sidewalk* to reflect a soft operator-provided preference for flying over grass rather than above pedestrian walkways.

We define the 2D environment cost map $\mathbf{C} \in \mathbb{R}^{H \times W}$ by taking the inner product of the semantic label vector at each pixel with the cost vector:

$$C_{ij} = \sum_{k=1}^n S_{ijk} c_k, \quad (3)$$

for all $(i, j) \in \mathcal{I}$. In compact notation, we write

$$\mathbf{C} = \mathbf{S} \cdot \mathbf{c}, \quad (4)$$

where “ $\cdot$ ” denotes contraction of the label dimension of $\mathbf{S}$ with the vector $\mathbf{c}$. Equivalently, each entry C_{ij} is the expected semantic traversal cost at image location (i, j) under the segmentation output.

In practice, both the motion planner and the runtime monitor operate on trajectories represented in the 2D planning frame. The planner uses the cost map $\mathbf{C}$ to generate candidate paths that avoid high-cost regions. For each candidate path, we view the sequence of waypoints and associated state variables (e.g. distance to the nearest obstacle or speed) as a discrete-time trace and evaluate its robustness with respect to previously defined STL specifications. This provides a quantitative measure of rule adherence for the planned trajectory. If a candidate path violates any hard constraint rules (negative robustness beyond acceptable limits), the affected portion of the environment cost map is re-weighted and the planner is invoked again for a path segment repair. This iterative process continues until a candidate path satisfies all hard constraint rules, at which point robot execution can begin.

As a concrete example, consider Rule 2 in Table II, *Obstacle Avoidance with Buffer*. Because this rule is time-invariant, it is encoded directly in the cost map by assigning high costs to obstacle-labeled pixels and to neighboring pixels within the desired safety margin. The planner then treats these regions as expensive to traverse, biasing paths away from obstacles before any temporal monitoring is applied.

2) *Planning-time STL screening (offline)*: We use STL specifications in a planning-time role by screening candidate trajectories before execution. Each candidate trajectory produced by the path planner is discretized into a trace of semantically relevant states, such as approaching stop signs, entering obstacle-dense regions, or transitioning from grass to path. In our initial implementation, this segmentation is configured manually based on task structure and map annotations; in future work, we aim to use VLMs to automatically identify and segment regions of interest by aligning key phrases in the STL rules (e.g., *grass* or *stop sign*) with image regions.

Given a candidate trace and the set of STL specifications, we evaluate robustness offline. If a segment of the trajectory is found to have negative STL robustness, indicating a violation, we invoke an incremental, repair-capable variant of our planner that selectively modifies only the offending segment of the path rather than replacing the entire path. This allows us to reuse non-violating segments of the path and reduce computational overhead while maintaining specification compliance.

If no feasible local repair can be found under the current specification, the system escalates to repairing a n increasingly greater subset of the candidate path, up to a full replan from the robot’s current state using the existing STL constraints.

As an example, consider Rule 3, *Grass-Exit Temporal Rule*, in Table II. Any candidate trajectory that remains in a grass-labeled region for more than 5 seconds after entering grass violates this specification and incurs a large penalty in the planning cost. When violation occurs, the corresponding segment is marked for repair, and the planner attempts to locally replace it with an alternative segment that meets specification.

3) *Execution-time STL monitoring (online)*: In addition to planning-time screening, we provide execution-time monitoring for online runtime assurance. During execution, the robot’s state over time is recorded as a discrete-time trace. The same STL specifications are evaluated online, producing robustness scores that indicate the degree to which each rule is currently satisfied as the robot navigates the environment.

In simulation, the executed trajectory should exactly follow a rule-satisfying plan. However, real robots in complex environments experience disturbances, perception errors, or modeling inaccuracies may cause deviations that lead to specification violations. When a violation is detected, the system flags the event and triggers replanning under the current STL constraints, using the updated environment cost map.

4) *Path Planning*: Path planning is performed using a classical search-based algorithm that optimizes a cost combining environment costs and penalties for violating STL rules. As the robot navigates through the environment, new visual information continuously refines the cost map, requiring the planner to adapt to an uncertain and dynamic environment that may include unexpected obstacles or terrain changes.

At each planning cycle, the robot updates its environment cost map $\mathcal{C}$ from the latest RGB observation using the VLM-based segmentation model described above. A sampling-based planner then proposes candidate trajectories in the 2D planning frame, optimizing a cost that combines $\mathcal{C}$ with penalties for STL rule violations.

We envision using a modified version of RRT known as RRT-X, an asymptotically optimal sampling-based motion planning algorithm for real-time navigation in dynamic environments with unpredictable obstacles [48], [49]. Assuming that the cost map and feasible regions frequently change with new VLM detections, mode switches, and other updates, a continuous-space, kinodynamic planner can repair a single tree faster than replanning from scratch. The motion planner maintains a continuous-space tree that is incrementally repaired as specifications or semantic labels change over time.

The integration of spatial and temporal reasoning within the same planning loop ensures that generated trajectories are both *valid* and *formally safe*.

C. State-Change: Normal versus Low Battery Mode

The architecture explicitly supports changes in robot operational states. We define two representative states: a **normal mode** with initial specifications and a degraded **low-battery mode** with stricter constraints on speed, obstacle clearance, and time-bounded behavior. Such mode changes illustrate how safety and performance requirements can adapt dynamically to external and internal system conditions. Table II lists the STL specifications for both modes and Figure 3 illustrates the state-change mechanism.

In this approach, we envision the transition to be manually triggered, though it could be automated through system state monitoring (e.g., battery triggers). When a state change occurs, inducing new specifications and desired robot behaviors, old specifications and cost maps are discarded. The robot pauses operation, recomputes its environmental cost map, and replans a path consistent with the new specifications. The state-switching mechanism demonstrates how tightening formal constraints affects the robot’s reasoning process, cost structure, and resulting trajectory.

D. Test & Evaluation

This architecture lends itself naturally to test and evaluation (T&E) analysis because STL provides quantitative measures of specification satisfaction through robustness metrics and a cost map provides insights into navigation efficiency and risk. Each execution generates a final cost, trajectory data, system logs, and corresponding robustness values for each specification, thereby enabling rule monitoring of both frequency and severity of specification violations.

Beyond treating violations as a binary phenomenon, we monitor for violation metrics such as the average robustness margin to safety constraints, average violation magnitude, and worst-case robustness over a run, and aggregate this data across multiple trials. These aggregate statistics facilitate post-hoc analysis to tune rule preferences, reconstruct where

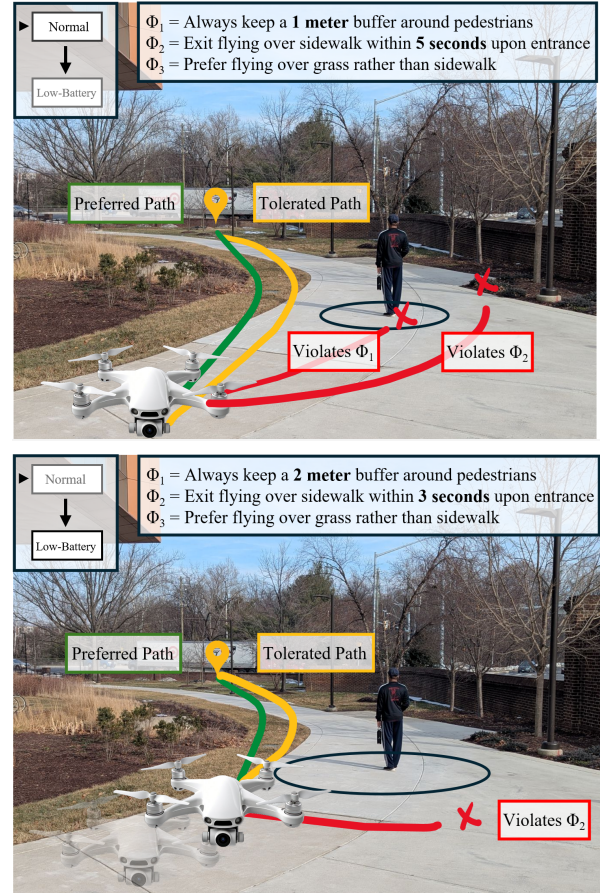


Fig. 3. State-dependent navigation illustrates normal versus low-battery mode. The finite-state controller switches specifications $\phi_1 - \phi_3$, which changes which candidate paths are treated as preferred (green), valid and less-preferred/tolerated (yellow), or violating (red).

and why violations occur, and a quantitative comparison between different system configurations.

Furthermore, by assigning priority levels to rules (e.g., “hard” constraints like safety and speed limits versus “soft” preferences such as path smoothness), we can evaluate trade-offs explicitly during operation. The system’s robustness and adaptability to specification changes can then be analyzed both online and offline using the same family of robustness scores and rule-violation metrics.

Together, these metrics define an interpretable T&E framework that clarifies how human language rules can be instantiated, monitored, and enforced within any perception-based robotic framework. This approach moves the navigation system toward a **correct-by-construction** design, providing a formal and empirically testable connection between human language rules, semantic understanding, and verified behavior of an autonomous robot.

IV. CONCLUSION

In this work, we proposed a theoretical framework that integrates human-provided safety rules into a Vision-and-Language Navigation stack for autonomous robots in novel environments. By grounding time-invariant Signal Temporal

Rule Description	Normal State	Low-Battery State
1. Global Speed Limit	The speed must always be below 5 kph. $G_{[0,\infty]} speed < 5kph$	The speed must always be below 3 kph. $G_{[0,\infty]} speed < 3kph$
2. Obstacle Avoidance with Buffer	Always keep 1 meter away from obstacles. $G_{[0,\infty]} dist.o \geq 1m$	Always keep 2 meters away from obstacles. $G_{[0,\infty]} dist.o \geq 2m$
3. Sidewalk-Exit Temporal Rule	Exit flying over sidewalk within 5 seconds upon entrance. $G_{[0,\infty]} status = sidewalk \rightarrow$ $F_{[0,5]} status = \neg sidewalk$	Exit flying over sidewalk within 3 seconds upon entrance. $G_{[0,\infty]} status = sidewalk \rightarrow$ $F_{[0,3]} status = \neg sidewalk$
4.1 Slow Before Stop	Slow to 5 kph within 5 seconds when nearing a stop sign, then stop. $G_{[0,\infty]} stop.obs \rightarrow F_{[0,5]} slow \wedge (F_{[0,5]} stop)$	Same as the normal state.
4.2 Pause At Stop	The robot must stop at the sign for at least 3 seconds. $G_{[0,\infty]} at.stop \rightarrow G_{[0,3]} speed = 0$	Same as the normal state.
5. Preference to Avoid Pedestrian Sidewalk	Prefer flying over grass rather than sidewalk. <i>Note: Soft preferences are encoded in the cost map.</i>	Preference removed in the low-battery state.

TABLE II

TABLE DESCRIBING RULES AND SOFT OPERATOR-BASED PREFERENCES FOR BOTH ROBOT STATES.

Logic (STL) specifications in VLM-based semantic segmentation, we obtain an environment cost map that encodes semantic preferences and safety margins. We then instantiate time-varying rules into a runtime assurance layer, which shapes candidate paths to satisfy a specified robustness threshold. In addition, we outline a theoretical T&E methodology that supports both online and offline verification of navigation systems.

Ongoing to this project is the instantiation of this framework into software for simulation-based experimentation. Future work aim to provide experimental hardware testing on a quadruped robot and UAV. We also plan to investigate automatically switching protocols from temporal logic systems for hybrid systems [50]. Another direction is to evaluate VLM components for resiliency under varying environmental and sensing conditions as part of the offline, post-hoc analysis of state change mechanisms. Using this framework to evaluate current VLM-based navigation systems on their ability to follow language-based rules during planning is also a potential avenue of exploration. Finally, we note that this approach can be incorporated into classical navigation stacks, complementing existing infrastructure to provide higher-fidelity safety reasoning for autonomous robot navigation.

REFERENCES

- [1] J. Ruz, O. Arevalo, G. Pajares, and J. M. D. La Cruz, "UAV Trajectory Planning for Static and Dynamic Environments," in *Aerial Vehicles*, T. Mung, Ed. InTech, Jan. 2009.
- [2] M. Saleem Sultan and M. Shahid Sultan, "Autonomous Systems in Unstructured Environments: AI Approaches for Robust Operation," *International Journal of Science and Research (IJSR)*, vol. 13, no. 8, pp. 1348–1355, Aug. 2024.
- [3] N. Wang, X. Li, K. Zhang, J. Wang, and D. Xie, "A Survey on Path Planning for Autonomous Ground Vehicles in Unstructured Environments," *Machines*, vol. 12, no. 1, p. 31, Jan. 2024.
- [4] J. Guiochet, M. Machin, and H. Waeselynck, "Safety-critical advanced robots: A survey," *Robotics and Autonomous Systems*, vol. 94, pp. 43–52, Aug. 2017.
- [5] J. Jiao, R. Geng, Y. Li, R. Xin, B. Yang, J. Wu, L. Wang, M. Liu, R. Fan, and D. Kanoulas, "Real-Time Metric-Semantic Mapping for Autonomous Navigation in Outdoor Environments," vol. 22, pp. 5729–5740. [Online]. Available: <https://ieeexplore.ieee.org/document/10620438/>
- [6] Z. Wei, S. Wang, K. Chen, and F. Wang, "ROS-Based Navigation and Obstacle Avoidance: A Study of Architectures, Methods, and Trends," *Sensors*, vol. 25, no. 14, p. 4306, Jan. 2025.
- [7] L. Ma, Y. Liu*, and J. Chen, "Using RGB Image as Visual Input for Mapless Robot Navigation," Apr. 2019.
- [8] T. Kim, S. Lim, G. Shin, G. Sim, and D. Yun, "An Open-Source Low-Cost Mobile Robot System With an RGB-D Camera and Efficient Real-Time Navigation Algorithm," *IEEE Access*, vol. 10, pp. 127 871–127 881, 2022.
- [9] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," 2021.
- [10] A. Singh, R. Hu, V. Goswami, G. Couairon, W. Galuba, M. Rohrbach, and D. Kiela, "FLAVA: A Foundational Language And Vision Alignment Model," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, Jun. 2022, pp. 15 617–15 629.
- [11] T. Wongpiromsarn, M. Ghasemi, M. Cubuktepe, G. Bakirtzis, S. Carr, M. O. Karabag, C. Neary, P. Gohari, and U. Topcu, "Formal Methods for Autonomous Systems," vol. 10, no. 3–4, pp. 180–407. [Online]. Available: <http://arxiv.org/abs/2311.01258>
- [12] E. Plaku and S. Karaman, "Motion planning with temporal-logic specifications: Progress and challenges," *AI Communications*, vol. 29, no. 1, pp. 151–162, Nov. 2014.
- [13] X. Li, Z. Serlin, G. Yang, and C. Belta, "A formal methods approach to interpretable reinforcement learning for robotic planning," *Science Robotics*, vol. 4, no. 37, p. eaay6276, Dec. 2019.
- [14] A. Manganaris, V. Giammarino, A. H. Qureshi, and S. Jagannathan, "Formal methods in robot policy learning and verification: A survey on current techniques and future directions," 2026. [Online]. Available: <https://arxiv.org/abs/2602.06971>
- [15] D. Shah, B. Osinski, B. Ichter, and S. Levine, "LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action," 2022.
- [16] Y. Du, T. Fu, Z. Chen, B. Li, S. Su, Z. Zhao, and C. Wang, "VL-Nav: Real-time Vision-Language Navigation with Spatial Reasoning," Mar. 2025.
- [17] K. Weerakoon, M. Elnoor, G. Seneviratne, V. Rajagopal, S. H. Arul, J. Liang, M. K. M. Jaffar, and D. Manocha, "Behav: Behavioral Rule Guided Autonomy Using VLMs for Robot Navigation in Outdoor Scenes," in *2025 IEEE International Conference on Robotics and*

Automation (ICRA). Atlanta, GA, USA: IEEE, May 2025, pp. 7044–7051.

- [18] Z. Li, S. Li, Z. Zhang, B. Li, and S. Zhou, “DV-VLN: Dual Verification for Reliable LLM-Based Vision-and-Language Navigation,” 2026.
- [19] L. Baird and S. Coogan, “Runtime Assurance from Signal Temporal Logic Safety Specifications,” in *2023 American Control Conference (ACC)*. San Diego, CA, USA: IEEE, May 2023, pp. 3535–3540.
- [20] R. Liu, A. Hou, X. Yu, and X. Yin. Zero-Shot Trajectory Planning for Signal Temporal Logic Tasks. [Online]. Available: <http://arxiv.org/abs/2501.13457>
- [21] P. Halder, H. Homburger, L. Kiltz, J. Reuter, and M. Althoff, “Trajectory Planning with Signal Temporal Logic Costs using Deterministic Path Integral Optimization.”
- [22] P. Kapoor, S. Vempala, and A. Kapoor. Logically Constrained Robotics Transformers for Enhanced Perception-Action Planning. [Online]. Available: <http://arxiv.org/abs/2408.05336>
- [23] B. Ye, J. Huang, Y. Liu, X. Qiao, and X. Yin. Bridging Perception and Planning: Towards End-to-End Planning for Signal Temporal Logic Tasks. [Online]. Available: <http://arxiv.org/abs/2509.12813>
- [24] J. Zhang, J. Huang, S. Jin, and S. Lu, “Vision-Language Models for Vision Tasks: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5625–5644, Aug. 2024.
- [25] Z. Li, X. Wu, H. Du, F. Liu, H. Nghiem, and G. Shi, “A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges,” 2025.
- [26] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 34 892–34 916.
- [27] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin, “Qwen2.5-VL Technical Report,” 2025.
- [28] H. Zhao, X. Puig, B. Zhou, S. Fidler, and A. Torralba, “Open vocabulary scene parsing,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2021–2029.
- [29] Y. Feng, Y. Liu, S. Yang, W. Cai, J. Zhang, Q. Zhan, Z. Huang, H. Yan, Q. Wan, C. Liu, J. Wang, J. Lv, Z. Liu, T. Shi, Q. Liu, and Y. Wang, “Vision-Language Model for Object Detection and Segmentation: A Review and Evaluation,” 2025.
- [30] J. Xu, S. De Mello, S. Liu, W. Byeon, T. Breuel, J. Kautz, and X. Wang, “GroupViT: Semantic Segmentation Emerges from Text Supervision,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, Jun. 2022, pp. 18 113–18 123.
- [31] S. Cho, H. Shin, S. Hong, A. Arnab, P. H. Seo, and S. Kim, “CAT-Seg: Cost Aggregation for Open-Vocabulary Semantic Segmentation,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Seattle, WA, USA: IEEE, Jun. 2024, pp. 4113–4123.
- [32] H. Shi, S. D. Dao, and J. Cai, “LLMFormer: Large Language Model for Open-Vocabulary Semantic Segmentation,” *International Journal of Computer Vision*, vol. 133, no. 2, pp. 742–759, Feb. 2025.
- [33] C. Huang, O. Mees, A. Zeng, and W. Burgard, “Visual Language Maps for Robot Navigation,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. London, United Kingdom: IEEE, May 2023, pp. 10 608–10 615.
- [34] S. Sahu, A. Singh, K. Nambiar, S. Saripalli, and P. B. Sujit, “Any-Traversal: An off-road traversability framework with VLM and human operator in the loop,” 2025.
- [35] D. Song, J. Liang, A. Payandeh, A. H. Raj, X. Xiao, and D. Manocha, “VLM-Social-Nav: Socially Aware Robot Navigation Through Scoring Using Vision-Language Models,” *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 508–515, Jan. 2025.
- [36] J. Ye, S. Papaioannou, and P. Kolios, “VLM-RRT: Vision Language Model Guided RRT Search for Autonomous UAV Navigation,” in *2025 International Conference on Unmanned Aircraft Systems (ICUAS)*, May 2025, pp. 633–640.
- [37] S. LaValle, “Rapidly-Exploring Random Trees: A New Tool for Path Planning,” Oct. 1998.
- [38] O. Maler and D. Nickovic, “Monitoring Temporal Properties of Continuous Signals,” in *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, Y. Lakhnech, and S. Yovine, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, vol. 3253, pp. 152–166.
- [39] A. Donzé, “On Signal Temporal Logic,” in *Runtime Verification*, A. Legay and S. Bensalem, Eds. Berlin, Heidelberg: Springer, 2013, pp. 382–383.
- [40] A. Donzé and O. Maler, “Robust Satisfaction of Temporal Logic over Real-Valued Signals,” in *Formal Modeling and Analysis of Timed Systems*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, K. Chatterjee, and T. A. Henzinger, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, vol. 6246, pp. 92–106.
- [41] J. A. Baier and S. A. McIlraith, “Planning with Preferences,” vol. 29, no. 4, pp. 25–36. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1609/aimag.v29i4.2204>
- [42] J. He, E. Bartocci, D. Ničković, H. Isakovic, and R. Grosu, “DeepSTL: From english requirements to signal temporal logic,” in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE ’22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 610–622.
- [43] Y. Mao, T. Zhang, X. Cao, Z. Chen, X. Liang, B. Xu, and H. Fang, “NL2STL: Transformation from Logic Natural Language to Signal Temporal Logics using Llama2,” in *2024 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE International Conference on Robotics, Automation and Mechatronics (RAM)*, Aug. 2024, pp. 469–474.
- [44] Y. Xu, J. Feng, and W. Miao, “Learning from Failures: Translation of Natural Language Requirements into Linear Temporal Logic with Large Language Models,” in *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*, Jul. 2024, pp. 204–215.
- [45] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, “NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models,” Mar. 2024.
- [46] T. Wongpiromsarn, U. Topcu, and R. Murray, “Formal Synthesis of Embedded Control Software: Application to Vehicle Management Systems,” in *Infotech@Aerospace 2011*. St. Louis, Missouri: American Institute of Aeronautics and Astronautics, Mar. 2011.
- [47] T. Luddecke and A. Ecker, “Image Segmentation Using Text and Image Prompts,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 7076–7086. [Online]. Available: <https://ieeexplore.ieee.org/document/9879551/>
- [48] M. Otte and E. Frazzoli, “RRT^X: Asymptotically optimal single-query sampling-based motion planning with quick replanning,” vol. 35, no. 7, pp. 797–822. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/0278364915594679>
- [49] —, “RRTX: Real-Time Motion Planning/Replanning for Environments with Unpredictable Obstacles.”
- [50] J. Liu, N. Ozay, U. Topcu, and R. M. Murray, “Synthesis of Reactive Switching Protocols From Temporal Logic Specifications,” *IEEE Transactions on Automatic Control*, vol. 58, no. 7, pp. 1771–1785, Jul. 2013.